AI时代软件工程重温清单

> 目标：不是回顾一遍传统软件工程名词，而是把**能将概率性 LLM 装进可控系统**的核心能力重新建立起来。
>
> 原则：**先 runtime，后框架；先系统，后技巧；先可控，后炫技。**

一、总目标

完成这份清单后，你应该能做到：

1. 理解一个简单 agent/runtime 的主逻辑
2. 分清哪些问题该交给 LLM，哪些问题该交给工程系统
3. 手写一个最小可用的 agent runtime
4. 看懂 LangGraph / Agents SDK / AutoGen 这类框架的核心抽象
5. 具备做一个简单但稳定 AI 应用的系统能力

二、复习优先级总览

P0：必须优先重温

1. 状态机与流程控制
2. 接口设计与结构化约束
3. 错误处理、恢复与持久化
4. 可观测性与调试
5. 上下文管理与信息压缩
6. 安全边界与权限控制

P1：紧随其后

7. 并发、异步与任务调度
8. 测试、评估与回归
9. 工具设计（Tool Design）
10. 数据与存储基础

P2：在有主线后再看

11. RAG 系统设计
12. 多 agent / graph orchestration
13. 框架源码与抽象层

三、详细复习清单

1. 状态机与流程控制（最高优先级）

为什么重要

AI 应用一旦进入多轮、工具调用、长任务，本质就变成了**状态化 workflow**。

要复习的点

- 有限状态机（FSM）
- 事件驱动模型
- 请求生命周期管理
- step loop / agent loop
- 终止条件
- 中断与取消（cancel / abort）
- timeout / retry / fallback
- deterministic workflow vs agentic workflow

你要能回答的问题

- 当前任务处于什么状态?
- 下一步为什么会发生?
- 什么时候应该结束，而不是继续“思考”?
- 用户点击停止后，系统如何中断?

最小练习

- 画出一个 agent loop 状态图
- 手写一个 while-loop agent runtime，支持：
  - 用户输入
  - 一次 tool call
  - 回填结果
  - step limit
  - 用户中断

完成标准

能用自己的话解释：

> agent 不是魔法，而是状态推进 + 工具执行 + 终止条件。

---

2. 接口设计与结构化约束

为什么重要

LLM 不稳定，所以接口必须更严格，而不是更松。

要复习的点

- API 设计基础
- JSON Schema / Pydantic / DTO
- 输入输出契约
- 参数校验
- fail-fast
- defensive programming
- structured output
- tool schema 设计

你要能回答的问题

- 模型输出为什么不能只靠“相信它”？
- tool call 的合法性由谁判断？
- 参数设计为什么直接影响调用成功率？

最小练习

- 为 3 个工具设计 schema：
  - search(query)
  - calc(expression)
  - read_file(path)
- 对 LLM 输出做一次 schema 校验与失败重试

完成标准

能写出：

- 一个清晰的 tool schema
- 一个 parser + validator
- 一个非法输出的处理逻辑

---

3. 错误处理、恢复与持久化

为什么重要

AI 系统不是“出错就重来”就能解决，尤其是长任务。

要复习的点

- retry 策略
- timeout
- 幂等性
- checkpoint / snapshot
- 崩溃恢复
- 补偿动作（compensating actions）
- durable execution
- 人工接管（human-in-the-loop）

你要能回答的问题

- 工具执行到一半失败，如何恢复？
- 哪些步骤可以重试，哪些不该重试？
- 长任务断掉后能否从中间恢复？

最小练习

- 给最小 runtime 加上：
  - max_steps
  - tool failure retry
  - checkpoint 保存当前 step
  - restart 后从 checkpoint 恢复

完成标准

能设计出一个简单任务的恢复机制，而不是只能“重新开始”。

---

4. 可观测性与调试

为什么重要

没有日志和 trace，AI 系统就是黑箱抽奖机。

要复习的点

- logging
- tracing
- metrics
- correlation id
- structured events
- 事件流回放
- 调试视角下的状态快照

你要能回答的问题

- 当前失败发生在哪一步？
- 是模型输出错了，还是工具炸了，还是 parser 有问题？
- 用户看到的结果和系统内部状态一致吗？

最小练习

- 给 runtime 加：
  - step_id
  - 每轮输入输出日志
  - tool call trace
  - 最终状态摘要

完成标准

出现错误时，你能在日志中定位：

- 第几步
- 调了什么工具
- 参数是什么
- 返回了什么
- 为什么终止

---

5. 上下文管理与信息压缩

为什么重要

AI 系统很容易上下文膨胀，最后变慢、变贵、变笨。

要复习的点

- working set
- session state
- 信息压缩
- 上下文摘要
- 短期记忆 vs 长期记忆
- 缓存
- 只保留必要状态

你要能回答的问题

- 哪些信息必须进入上下文?
- 哪些只需要进日志，不需要回填模型?
- 长任务如何避免上下文不断变肥?

最小练习

- 为一个 10 步任务设计上下文裁剪策略
- 区分：
  - 当前任务必要信息
  - 历史摘要
  - 日志/审计信息

完成标准

能解释：

> 不是所有中间结果都该喂回模型。

---

6. 安全边界与权限控制

为什么重要

AI 系统一旦能调工具，就会接近真实世界风险。

要复习的点

- least privilege
- sandbox
- 输入校验
- 输出过滤
- auth / authz
- secret handling
- 风险分级
- 审计日志

你要能回答的问题

- 哪些工具可以直接执行?
- 哪些工具必须人工确认?
- 模型如果输出危险命令，谁来拦?

最小练习

- 为 5 个工具做权限分级：
  - 只读
  - 低风险写操作
  - 高风险执行
- 设计一个人工确认机制

完成标准

能把“模型想做什么”和“系统允许做什么”彻底分开。

---

7. 并发、异步与任务调度

为什么重要

AI 应用会遇到并发工具、流式输出、后台执行、取消传播等问题。

要复习的点

- async/await
- 任务调度
- 并发与串行依赖

- backpressure
- cancellation propagation
- 队列与 worker

最小练习

- 让两个独立工具并发执行
- 支持用户中断一个长任务

---

8. 测试、评估与回归

为什么重要

AI 系统不能只靠“看起来像能用”。

要复习的点

- unit test
- integration test
- end-to-end test
- golden set
- regression
- failure injection
- 行为评估
- 工具调用成功率评估

最小练习

- 为一个 agent 设计 20 条测试样例
- 记录：
  - 是否调用正确工具
  - 是否返回合法结构
  - 是否在最大步数内完成

---

9. 工具设计（Tool Design）

为什么重要

工具设计直接决定 agent 的稳定性和效率。

要复习的点

- 工具粒度
- 工具命名
- 参数命名
- 返回值结构化
- provider-specific schema 兼容
- tool result 压缩

最小练习

- 把 6 个过细工具重构为 2 到 3 个更合理工具
- 对比调用轮数与稳定性

---

10. 数据与存储基础

为什么重要

很多 AI 应用最后都会落到：会话、文档、日志、记忆、索引、任务状态。

要复习的点

- session store
- task store
- document store
- cache
- vector store 的边界
- 一致性与新鲜度
- 数据生命周期

最小练习

- 为一个简单 AI 助手设计最小数据模型：
  - messages
  - tasks
  - tool_logs
  - summaries

四、AI时代软件工程的学习顺序建议

第 1 阶段：主逻辑

- 状态机
- API/Schema
- Tool registry
- Agent loop
- Stop condition

第 2 阶段：可靠性

- Retry / timeout
- Logging / tracing
- Checkpoint / resume
- Validation / guardrails

第 3 阶段：规模化

- Async / concurrency
- Evaluation / regression
- Context compaction
- Persistence

第 4 阶段：框架理解

- OpenAI Agents SDK
- LangGraph
- AutoGen Core
- MCP

五、建议的源码阅读顺序

第一批：轻抽象，先看骨架

1. OpenAI Agents SDK
    - 看 primitives
    - 看 tools / handoffs / guardrails / tracing

2. LangGraph
    - 看 state graph
    - 看 persistence / checkpoint / tool node

第二批：多 agent 与事件驱动

3. AutoGen Core
    - 看 event-driven / actor model
    - 看 agent chat 是怎么建立在 core 上的

第三批：补充协议层

4. MCP 相关 SDK / server / client
    - 看工具互联和宿主协议

六、最值得做的 5 个练习项目

项目 1：最小 Tool-Calling Runtime

要求：

- 单 agent
- 3 个工具
- schema 校验
- step limit
- retry
- logging

项目 2：带中断与恢复的长任务 Agent

要求：

- checkpoint
- resume
- cancel
- 最终状态追踪

项目 3：带上下文压缩的文件分析助手

要求：

- 多轮文件读取
- 中间摘要
- working set 控制

项目 4：受权限约束的执行型 Agent

要求：

- 只读/写/危险操作分级
- 人工确认
- 审计日志

项目 5：最小评估系统

要求：

- 测试集
- 成功率统计
- 错误类型分类
- 回归对比

七、复习时要避免的误区

误区 1：先沉迷框架

框架是封装，不是本体。

误区 2：把 RAG 当主线

RAG 是能力模块，不是 runtime 骨架。

误区 3：迷信多 agent

很多时候只是把简单问题复杂化。

误区 4：只看 prompt

真正难的是系统控制，不是多写几句提示词。

误区 5：没有日志就调试

这基本等于祈祷。

八、最终自检问题

复习完后，你应该能独立回答：

1. 一个 agent runtime 的最小骨架是什么？
2. 为什么 tool call 不能只靠 prompt 保证？
3. 哪些逻辑应该 deterministic，哪些才交给 LLM？
4. 长任务失败后怎么恢复？
5. 为什么上下文不能无限累积？
6. 如何让用户中断真正传递到系统执行链路？
7. 如何定位一次工具调用失败的根因？
8. 为什么优秀 AI 产品很多优势其实来自 runtime 和封装？

九、最后的判断标准

如果你重温完软件工程后，依然只会：

- 接 API
- 堆框架
- 做 demo

那说明你复习错了方向。

如果你能开始：

- 设计状态
- 限制复杂度
- 控制上下文
- 约束工具
- 恢复失败
- 观察行为

那说明你真的进入了 AI 时代的软件工程主线。

十、一句话版

> AI时代最该复习的软件工程，不是“怎么更快写业务”，而是“怎么把不稳定智能装进稳定系统”。